SYSTEMS MANAGEMENT



Creating Aggregation Pipelines to Conquer Large Telemetry Challenges

Jim Harford

System Architect, Broadcom

OCP PLATINUM® Member

OCP SCIM Workstream

- “SCIM” = Scalable Cloud Infrastructure Management (part of Hardware Management)
- Evolution of previous CSM (Cloud Service Model) workstream
- SCIM’s charter is to address hardware management of large data centers (100,000 nodes or more), creating documents in the following areas:
  - Architectures, protocols, and data models that communicate hardware management information across data center networks;
  - Analysis of existing communication mechanisms and interacting with standards bodies to improve the effectiveness of those mechanisms for hardware management at data center scale;
  - Management challenges associated with new technologies (e.g. AI/ML, RDMA, PCIe/CXL, DDR, quantum computing); and
  - Predictive models that forecast how anticipated aspects of hardware systems affect the financial performance of the data center.

Our Approach to Telemetry Standards

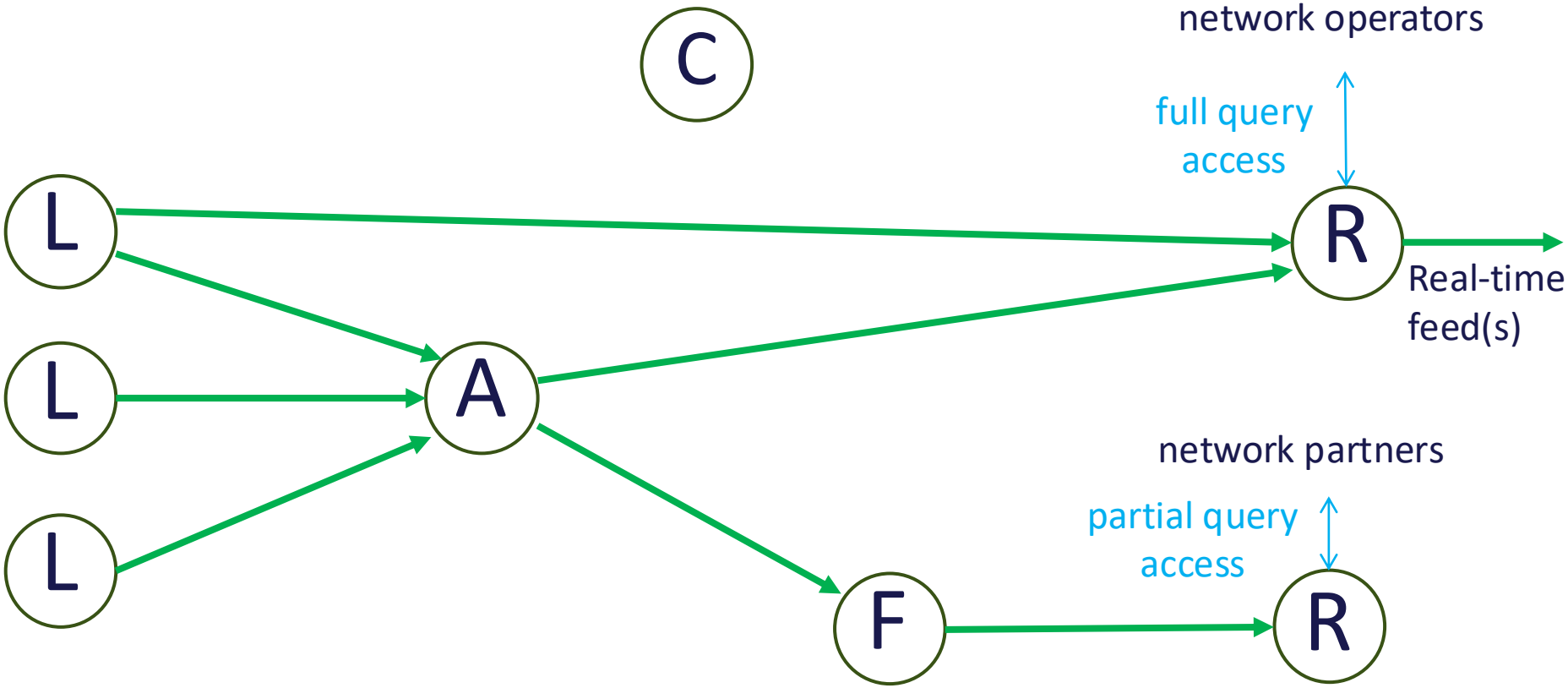
- Many protocols, schemas, and technologies to choose from!
  - Redfish, YANG, Open Telemetry, Prometheus, …
- Many naming choices for individual metrics
- In our desire to bring order to the chaos, SCIM intends to follow these steps:
  1. Create a high-level, *protocol-agnostic* architecture for telemetry
  2. Map (as best we can) our architecture into the various protocols (see 1st bullet)
  3. Work with the responsible standards groups and open-source communities to improve their respective protocols
  4. Encourage the standardization of names for individual metrics of interest

Structural Components

We break the telemetry pipeline into *nodes*, which can be characterized as follows:

- **Leaf nodes** - endpoints that emit streams of telemetry information
- **Aggregation nodes** - intermediate nodes that aggregate telemetry streams received into a smaller number of output telemetry streams
- **Filtering nodes** - intermediate nodes that remove certain information from telemetry streams, to prevent sensitive information from propagating
- **Reporting nodes** - endpoints that store and report telemetry information of interest to authorized personnel and systems
- **Control node** - a single compute entity (or confederation) that coordinates the flow of telemetry information from leaf nodes towards reporting nodes

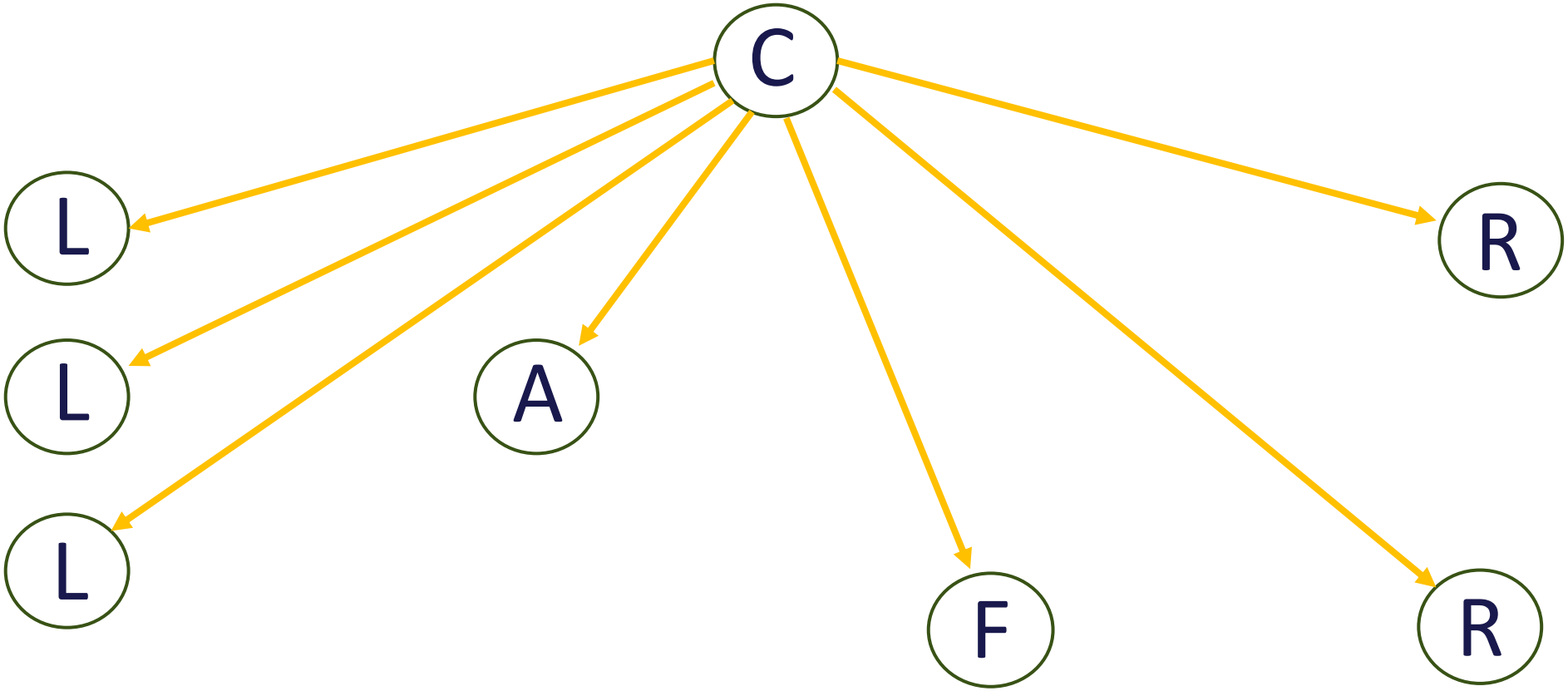
Telemetry Data Streaming Between Nodes



Node Types:

L = Leaf
A = Aggregation
F = Filtering
R = Reporting
C = Control

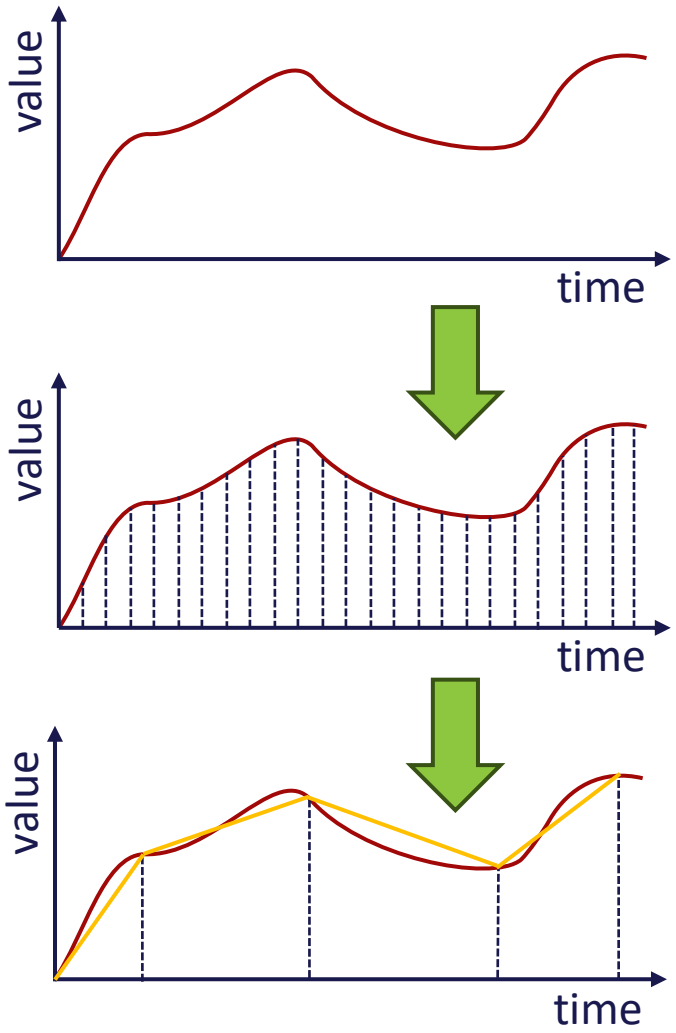
Setup of Telemetry Streams



Node Types:
L = Leaf
A = Aggregation
F = Filtering
R = Reporting
C = Control

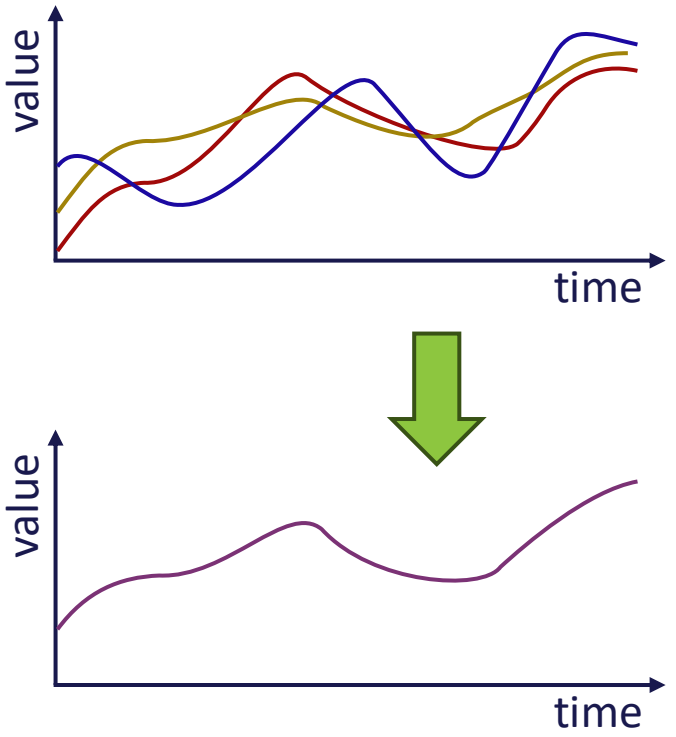
Temporal Aggregation

- ***Goal:*** compress data in the time dimension
- ***Methods:*** (1) continuous to discrete time domain, (2) lower sampling frequency, (3) leverage unequal inter-sample durations
- ***Benefits:*** (1) save storage, (2) save bandwidth, (3) save processing time
- ***Other:*** most common



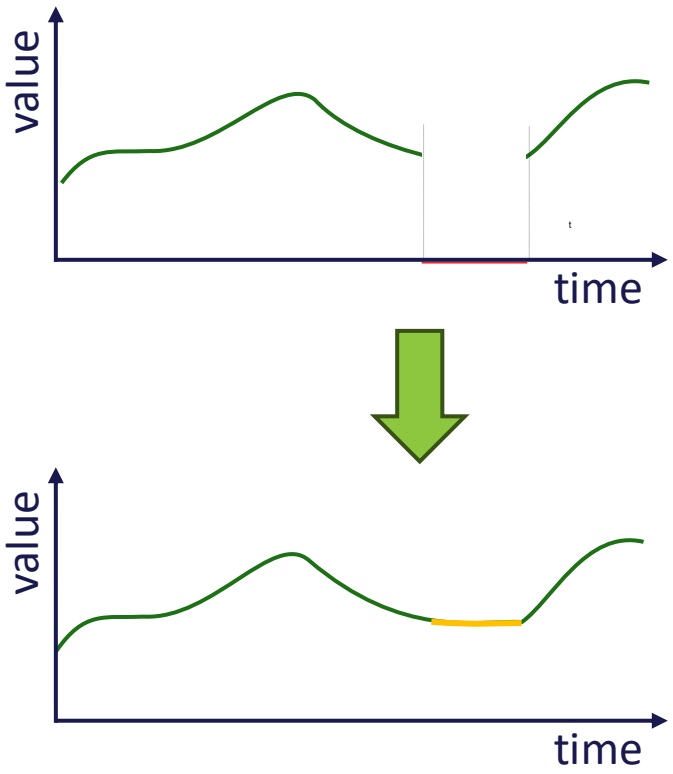
Spatial Aggregation

- ***Goal:*** combine data from different sources:
  - Across components within a server
  - Across servers within a rack
  - Across racks within a room
  - Across rooms within a data center
- ***Methods:*** (1) average the inputs, (2) determine largest input, (3) determine smallest input
- ***Benefits:*** (1) see the "big picture", (2) save bandwidth, (3) save processing time
- ***Other:*** necessary to scale up & compare inputs



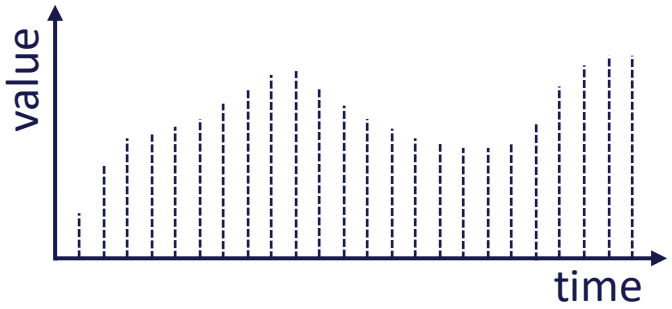
Contextual Aggregation

- ***Goal:*** include relevant data, exclude extraneous or misleading data
- ***Methods:*** (1) ignore bad sources, (2) “patch” missing data, (3) context tags for operational status, tenant, workload, user-defined, etc.
- ***Benefits:*** (1) more accurate, (2) more flexible, (3) more relevant, (4) create control signals
- ***Wide Variety of Use Cases:***
  - Exclude machines undergoing routine maintenance
  - Vendor study, tenant utilization study
  - Create & study ad-hoc clusters of sources
  - Capture pre & post conditions around some anomaly



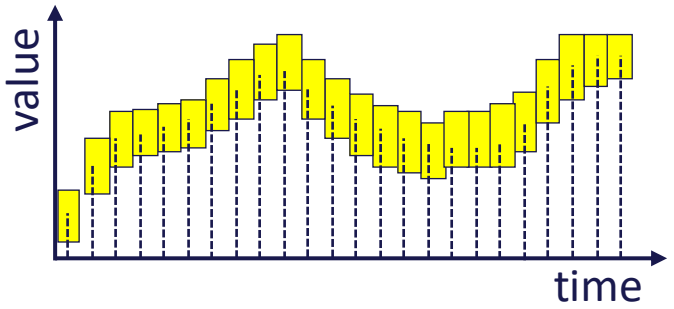
Regular Time Series

- A sequence of *points* recorded over time
- Current value of some observable (vertical axis) at equally spaced points in time (horizontal axis)
- A form of discrete-time data from either discrete or continuous sources
- High availability of mathematical tools:
  - Fourier-based transforms & analysis
  - Curve fitting
  - Filtering
  - Prediction
  - Segmentation



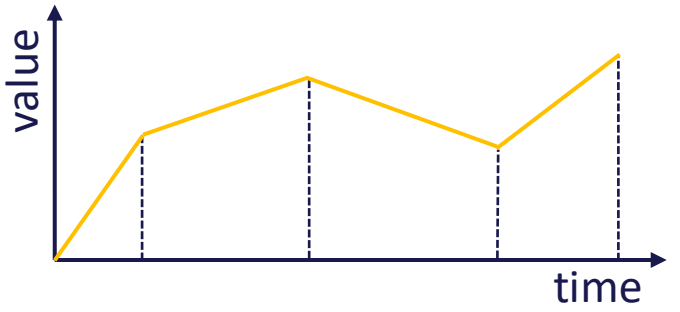
Stats Time Series

- A sequence of *intervals* recorded over time
- Multiple statistical measurements (vertical axis) for equally spaced time intervals (horizontal axis)
  - minimum
  - maximum
  - average
- Offers much better insight into the signal, as compared with a regular time series
- Each individual statistical measurement could be considered a regular time series



Irregular Time Series

- Similar to a regular time series, but there is no expectation of equally spaced points in time
- Can be very efficient to represent signals with predominantly long trend lines
- Can be very efficient to segment a signal into areas with different trend lines
- Very useful for anomaly detection and event-based telemetry.
- Each data point has a timestamp, which may facilitate correlation with other time series.



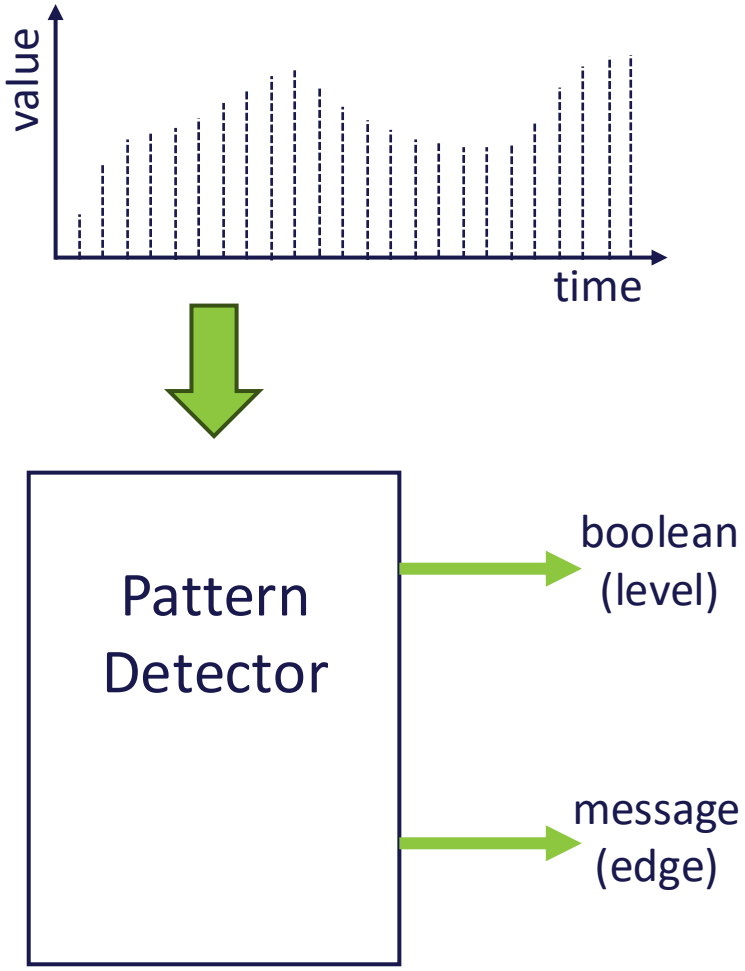
Conversions Among Time Series Types

- Within a node where aggregation is taking place, it is common to convert among different representations of a time series, in order to leverage the respective advantages of each one.

Conversion Type	Motivations
Regular to Regular	resample to normalize against another time series; reduce bandwidth
Stats to Stats	resample to normalize against another time series; reduce bandwidth
Regular to Stats	enhance statistical information; reduce noise
Stats to Regular	leverage math transforms (e.g. Fourier); feed to next stage in pipeline
Regular to Irregular	reduce bandwidth; helpful in some pattern detection
Irregular to Regular	leverage math transforms (e.g. Fourier); feed to next stage in pipeline

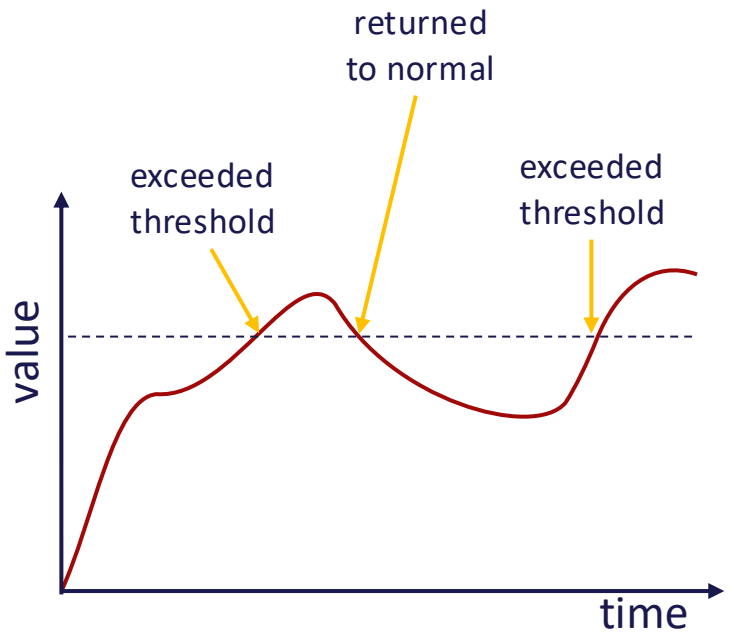
Simple Pattern Detectors

- ***Purpose:*** indicate the presence of some pattern of interest embedded within an input signal
- ***Input:*** a time series - regular, stats, or irregular
- ***Output:*** indicates presence and later absence of specified pattern by (1) continuous boolean signal and/or (2) edge notifications
- ***Parameters:*** (1) release hysteresis – time delay before indicating later absence of pattern (minimizes thrashing around presence/absence)



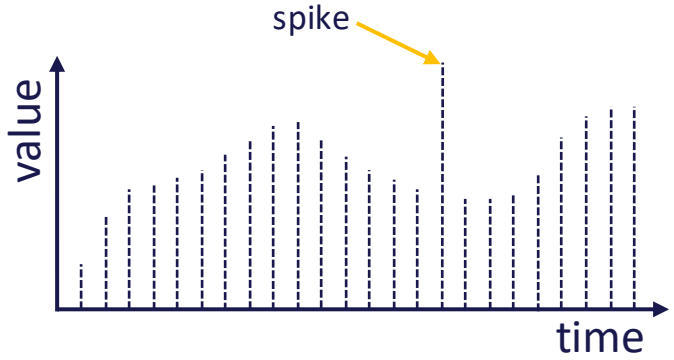
Threshold Pattern Detection

- ***Criteria:*** transition from “more normal” region to “less normal” region of values
- ***Parameters:*** (1) threshold value, (2) direction
- ***Example Use Cases:***
  - Temperature unsafe, too much power used
  - Error count on network packets received
  - Utilization of CPU low
  - This is the most common detection
    - Sometimes, it is built into hardware.



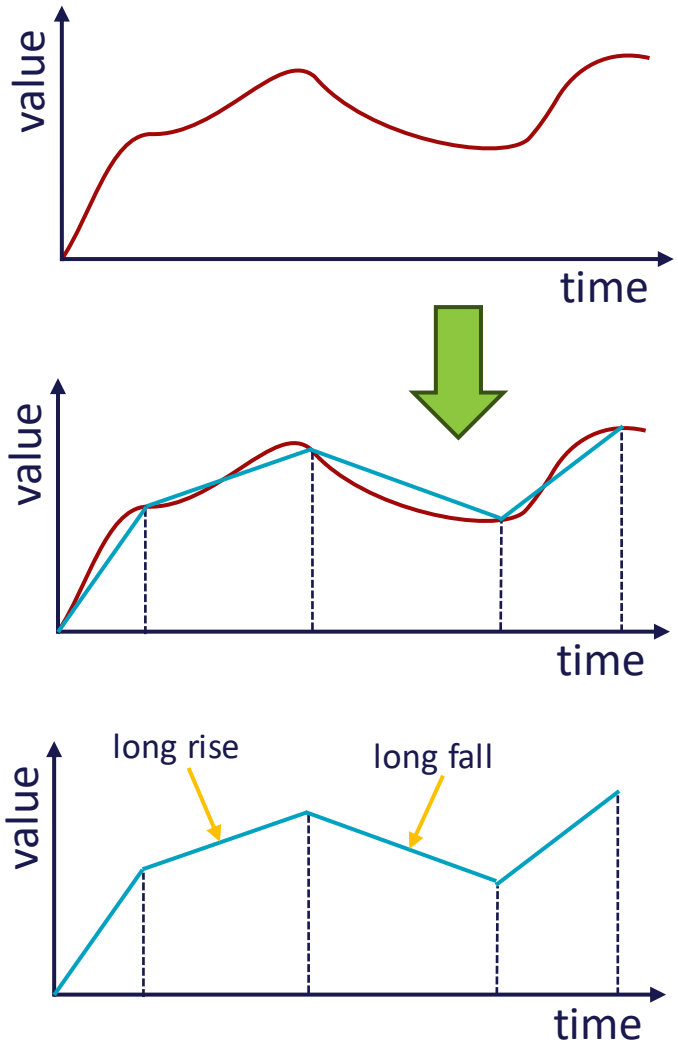
Spike Pattern Detection

- ***Criteria:*** significant, transient excursion from recent average value
- ***Parameters:*** (1) threshold, (2) direction, (3) minimum duration, (4) maximum duration
- ***Example Use Cases:***
  - Voltage spike when a machine resets
  - Network traffic has dropped
  - Temperature overshoot, but cooling loop has not adjusted quickly enough



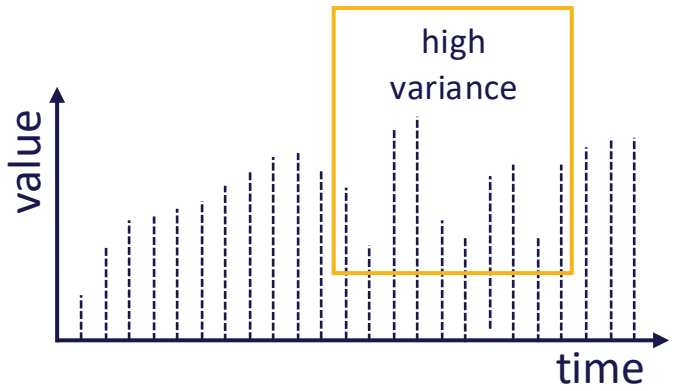
Trend Pattern Detection

- ***Criteria:*** sustained, average, increase or decrease
- ***Parameters:*** (1) direction, (2) minimum duration, (3) input moving average width
- ***Example Use Cases:***
  - Memory leaks
  - Temperature drift upwards (caused by liquid cooling leaks)
  - Power drift on pluggable transceiver TX laser



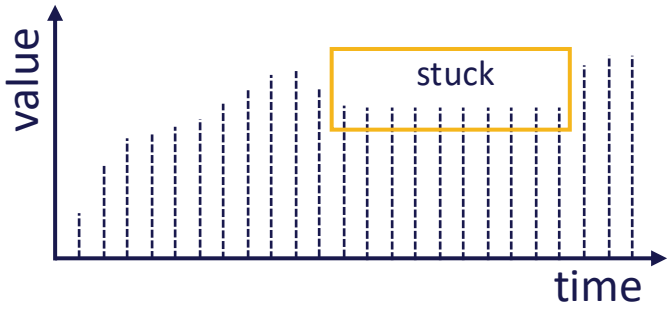
Variance Pattern Detection

- ***Criteria:*** signals high variance in values
- ***Parameters:*** (1) variance threshold, (2) input moving average width
- ***Example Use Cases:***
  - Voltage may signal bad capacitors, power supply
  - Disk IO jitter
  - High CPU/GPU variance may signal that workloads need to be rebalanced or optimized



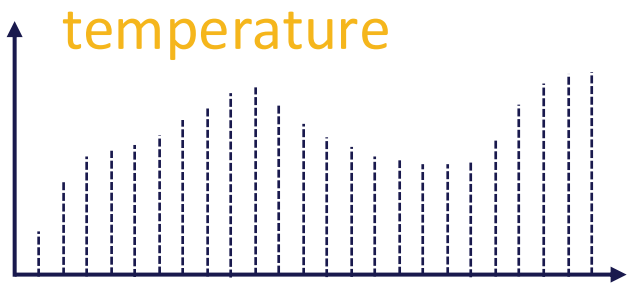
Stuck Pattern Detection

- ***Criteria:*** the repetitive reading of a single value, which is thought to be very unlikely
- ***Parameters:*** (1) minimum duration, (2) is zero a valid repetitive reading?
- ***Example Use Cases:***
  - Detection of bad sensor
  - Detection of broken path to/from sensor
  - Missing component in a closed (feedback) loop



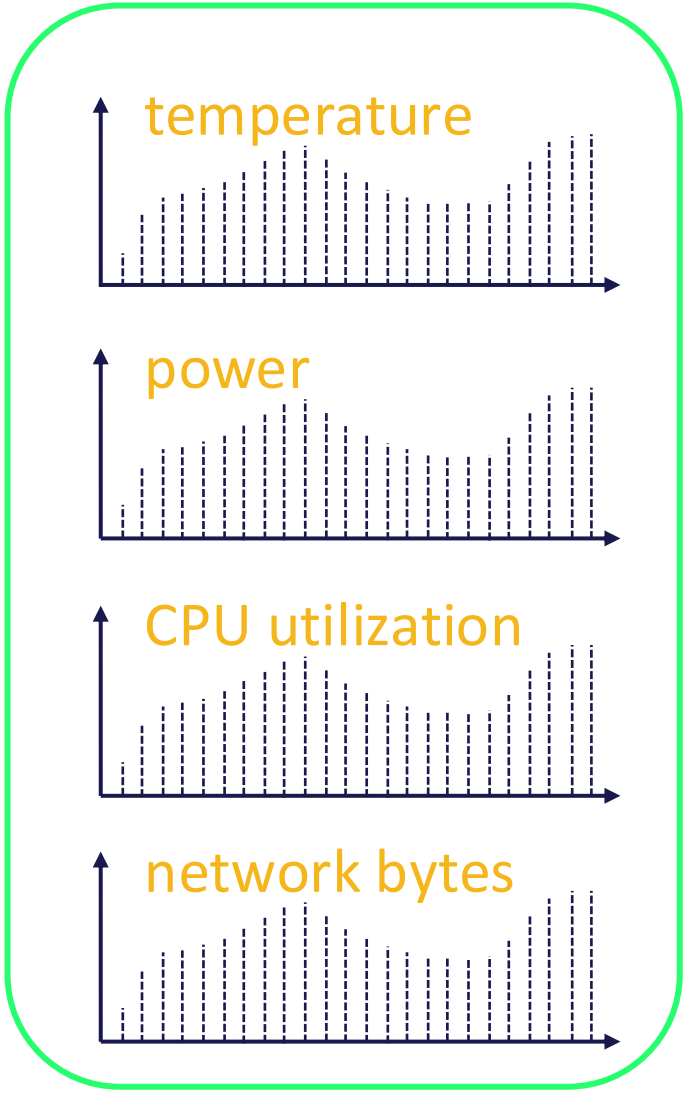
Metric

- ***Definition:*** a standard measurement used to assess & quantify some phenomenon of interest
- ***Properties:***
  - name : string
  - local_path : string
  - virtual_index : int
  - meter_type : GAUGE, COUNTER
  - value_type : integer, unsigned integer, floating point
  - value_width : 1, 2, 4, or 8 bytes
  - units : string (UCUM standard)
  - measurement_scale : ORDINAL, INTERVAL, RATIO
  - metric_ID : *local handle* at aggregation node



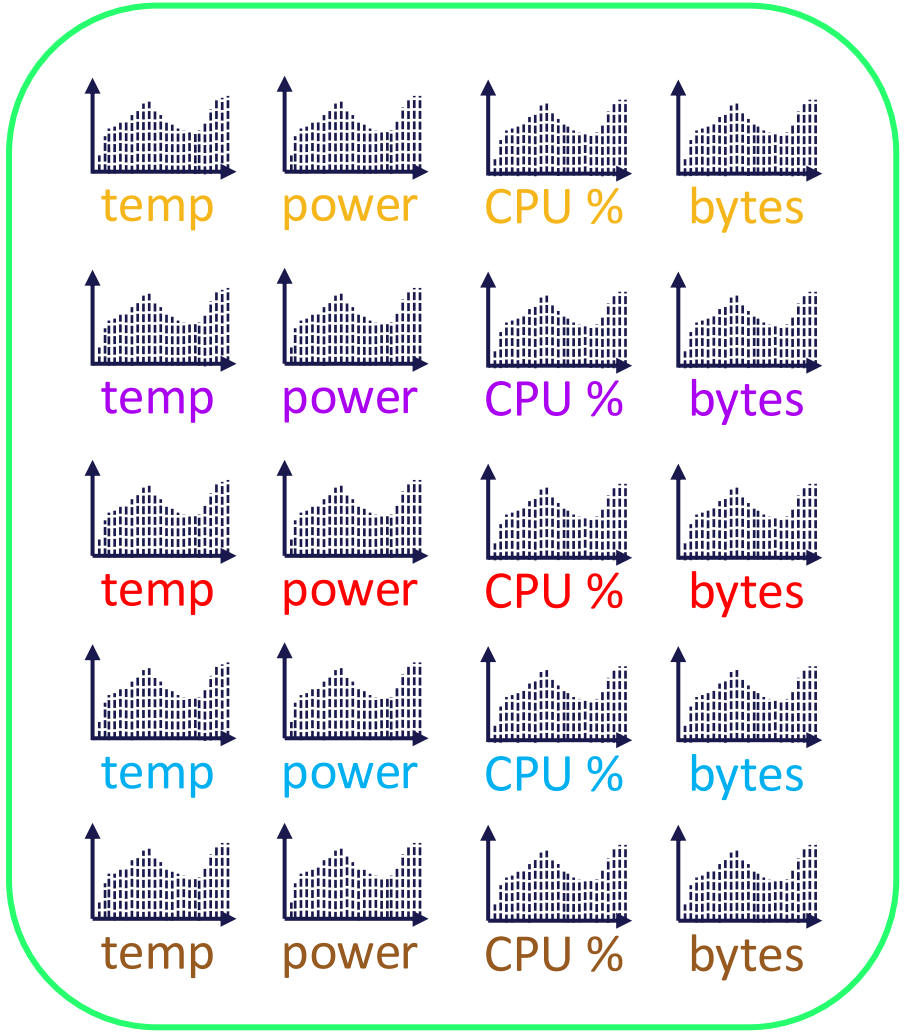
Bundle

- ***Definition # 1:*** a set of metrics that are included in a *stream* from a single node
- ***Definition # 2:*** a set of temporally parallel time series from a single node
- ***Limitations:*** regular and stats time series only
- ***Properties:***
  - reference_timestamp
  - frequency : real
  - source : sending node
  - metric_IDs[1..N]
  - metric_values[1..N]
  - bundle_ID : *local handle* at aggregation node



Super Bundle

- ***Definition:*** a set of "*matching bundles*" from multiple nodes
- ***Sub-Definition:*** "*matching bundles*" means bundles with matching descriptions of metric names, formats, and respective ordering
- ***Limitations:*** regular and stats time series only
- ***Properties:***
  - reference_timestamp
  - frequency : real
  - bundle_IDs[1..M]
  - time_series_bundle[1..M]
  - super_bundle_ID : *local handle* at aggregation node



Finding Outliers (1)

- We define a *source value pair* to be an ordered pair (source : node, metric_value : real)
- We define a *multi-source metric snapshot* to be a list of the metric value from different source nodes, with the properties:
  - metric_ID
  - timestamp
  - source_values[1..N] : source_value_pair

Server	Outlet Temp (C°)
Christy	41
Bob	60
Elise	36
Joel	44
Deborah	40
Alice	25
Joshua	42
Eve	50
Matthew	39
James	40

Finding Outliers (2)

- We define a *source rank pair* to be an ordered pair (source : node, metric_rank : ordinal)
- We define a *multi-source metric ranking* to be a list of the metric ranks from different source nodes, with the properties:
  - metric_ID
  - timestamp
  - source_ranks[1..N] : source_rank_pair

Server	Temperature Rank
Christy	5
Bob	1
Elise	9
Joel	3
Deborah	6
Alice	10
Joshua	4
Eve	2
Matthew	8
James	7

Finding Outliers (3)

- We define a process, *metric ranker*, by which a *multi-source metric snapshot* is converted into a *multi-source metric ranking*
- Now we can define the following operations on the *multi-source metric ranking*:
  - get_source_by_rank (ordinal) : source
  - get_rank_by_source (source) : ordinal
  - get_percentile_by_source (source) : int
- *Use cases:*
  - Identify the best performing source nodes
  - Identify the worst performing source nodes

Server	C°	Rank
Christy	41	5
Bob	60	1
Elise	36	9
Joel	44	3
Deborah	40	6
Alice	25	10
Joshua	42	4
Eve	50	2
Matthew	39	8
James	40	7

Hottest servers are Bob & Eve.
Coolest servers are Alice & Elise.

The Big Ideas

- Aggregation allows scale up and scale out to hyperscale-size data centers
- Perform pattern detection and alert generation closer to the source
  - Avoid latency and the cost of propagating telemetry data
- Abstraction of these aggregation mechanisms should promote interoperability
  - Aggregation helps combine sources of data from different vendors into actionable results
  - Aggregation can condition the data for multiple consumers (including external consumers)

Next Steps

- Validate these ideas with a minimal proof-of-concept implementation
- Need a way to specify one or more representations of how to combine these aggregation mechanisms into a telemetry pipeline
- Include these ideas in SCIM's forthcoming specification on streaming telemetry
- Continue our discussions with members of the Redfish Forum to reach points of alignment with them in their continuing work on telemetry

Thank You!

- Where to find additional information (URL links)
  - opencompute.org/projects/scalable-cloud-infrastructure-management
  - github.com/opencomputeproject/OCP-HM-SCIM-Scalable_Cloud_Infrastructure_Management/wiki

